Wellbore Viewer — React Architecture, Entra ID Auth & Boilerplate

Oct 19, 2026 · @Sony Lazarus

Rebuild `wellbore-trajectory-react` on the boilerplate attached to this guide: a typed, feature-sliced React app where the drilling simulation runs outside React renders and every user signs in through Microsoft Entra ID.

Purpose

This guide is the follow-up to the code review of the current repo. The review lists what is broken; this guide sets the target and gives you a working starting point.

What you are asked to do:

1. Read *Architecture principles* and *State and 3D rendering pattern* before writing code.
2. Start from the boilerplate (`wellbore-viewer-boilerplate.zip`), not the current repo. It already builds, lints, type-checks and passes its tests.
3. Register the app in Entra ID as described in *Entra ID setup*, fill in `.env.local`, and confirm sign-in works.
4. Port features from the reference HTML one feature folder at a time, following *Migration*.

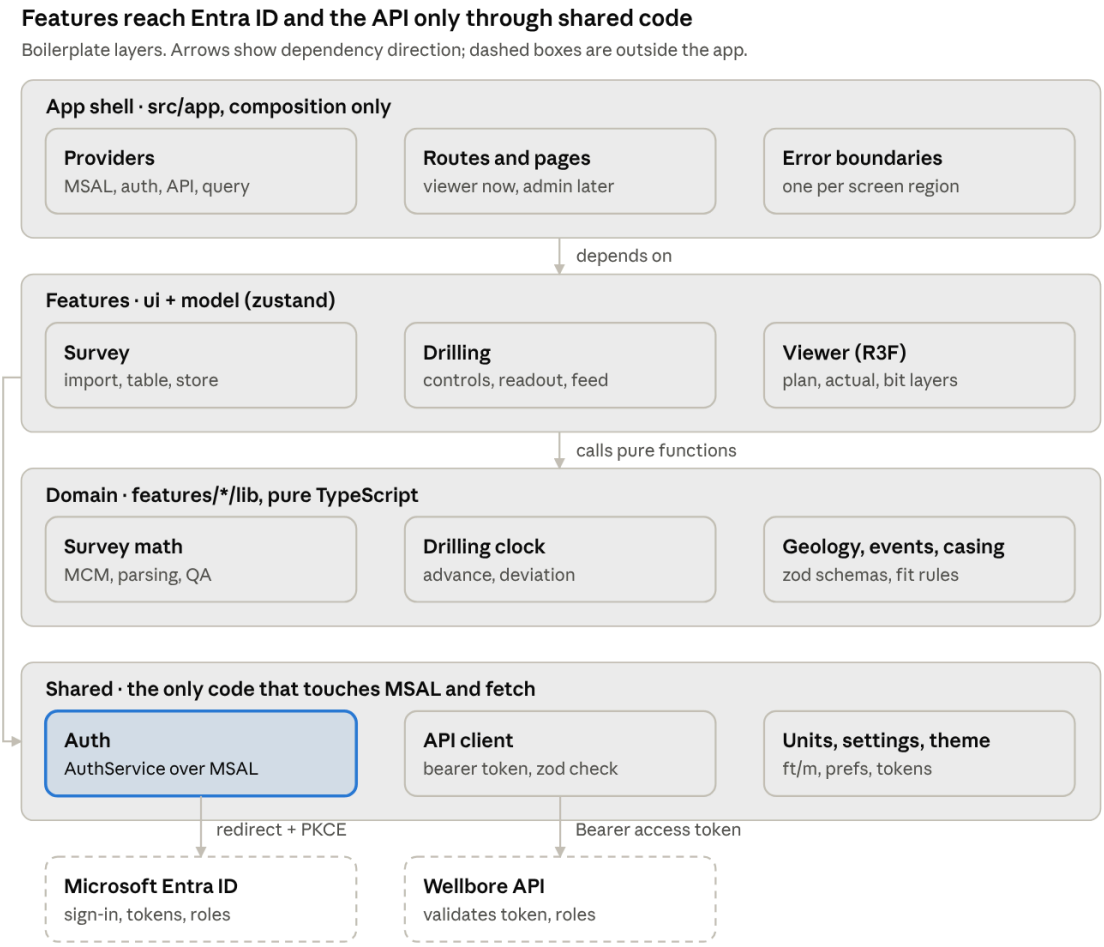
The reference for behaviour stays *Multilateral Trajectory Viewer (2).html*. When this guide and the HTML disagree on behaviour, the HTML wins; when they disagree on code structure, this guide wins.

Architecture principles

Eight rules. A pull request that breaks one needs a written reason in its description.

1. **Organise by feature, not by file type.** `features/geology` owns its store slice, its pure logic and its UI. No global `components/` or `utils/` dumping ground.
2. **Domain code is pure TypeScript.** `features/*/lib` never imports `react`, `three` or a parser. It runs and is tested in Node.
3. **Animation never goes through React state.** The bit moves in `useFrame` by reading the store directly. Panels subscribe with a 200 ms throttle. No `setState` inside an animation loop.
4. **3D scene = React Three Fiber components.** One component per layer (plan, actual, sleeve, events, casing, scale). Geometry is rebuilt only when its data version changes, never per frame.
5. **One internal unit.** Store feet. Convert only in `shared/units` hooks at the UI edge.
6. **Validate at every boundary with zod.** File imports, form input, persisted data and API responses all pass a schema before entering the store.
7. **Authentication is a shell concern.** Feature code never calls MSAL. It reads the user and roles from `useAuth()` and calls APIs through `shared/api`, which attaches the token.
8. **Front-end role checks are for display only.** The API enforces every permission. Hiding a button is not security.

At a glance



boilerplate layers · 4 layers, 2 external services

Pages compose features; features call pure domain functions and reach identity, the network and units only through `shared/`. The line on the left is features using shared code directly. Domain code depends on nothing but `zod`, which is why it can be tested without a browser.

Folder structure

Each feature owns its pure logic (`lib/`), its state (`model/`) and its UI (`ui/`), and exposes one public `index.ts`. ESLint blocks deep imports into another feature and blocks MSAL outside `shared/auth`.

```
src/
  main.tsx                validate env → init MSAL + handle redirect → render
  app/                    App (routes), Providers, ViewerPage, AppHeader — composition only
  features/
    survey/      lib/  mcm.ts · parseSurvey.ts · parse.worker.ts · qa.ts · schema.ts
                 model/ surveyStore.ts     ui/ SurveyImport · SurveyTable
    drilling/    model/ engine.ts · drillingStore.ts · deviation.ts
```

```
                ui/ DrillControls · BitReadout · LiveFeed
    viewer/     lib/  scene.ts            ui/ Viewport · CameraRig · layers/ (Plan,
Actual, Bit, Grid)
    geology/    lib/  formationAt.ts      model/ types.ts (zod)        ← UI to port
    events/     model/ types.ts (21 event types, zod)                  ← UI to port
    construction/ lib/ rules.ts (holeAt, fit checks) model/ types.ts    ← UI to port
  shared/
    auth/       AuthService interface · entraAuth (MSAL) · mockAuth · useAuth ·
<Authorized> · roles
    api/        createApiClient (bearer token, correlation id, zod) · useApi
    config/     env.ts (zod-validated VITE_* variables)
    units/      ft ↔ m, DLS °/100 ft ↔ °/30 m, useUnits()
    settings/   unit + theme preferences (persisted)
    theme/      tokens.css (light/dark, also read by the 3D scene) · app.css
    lib/ ui/    useThrottledStore · ErrorBoundary · Segmented
public/samples/  the reference HTML's 3-lateral plan and actual surveys as CSV
docs/            ARCHITECTURE · AUTH · MIGRATION · QUALITY
```

Folder	May import	Must not import
`features/*/lib`	own `lib`, `zod`	React, three.js, stores, other features' UI
`features/*/model`	own `lib`, other features' `index`, `shared`	UI components
`features/*/ui`	own `model` + `lib`, other features' `index`, `shared`	MSAL, another feature's internals
`shared/*`	`shared` only	any feature
`app/`	everything	—

State and 3D rendering pattern

The bit MD changes every frame but React re-renders at most a few times a second. Three mechanisms make that work, and every new feature uses the same three.

Who needs the bit MD	How it reads it	React renders
`BitLayer` (moves the bit)	`useDrilling.getState()` inside `useFrame`	Never
`ActualLayer` (drilled tube)	Selector returning `stationsReached(actual, md)` — a count	Only when the bit crosses a station
Readout, controls, table	`useThrottledStore(useDrilling, s => s.md, 200)`	≤ 5 times a second

1. The clock runs in the render loop, as a pure function.

```
// features/drilling/model/engine.ts — unit-tested, no React
export function advance(state: ClockState, dtSeconds: number, tdFt: number):
ClockState {
  if (!state.playing || tdFt <= 0) return state;
  const dt = Math.min(0.1, Math.max(0, dtSeconds)); // a backgrounded tab must not
```

```
jump the bit
  const md = Math.min(tdFt, state.md + state.speedFtPerS * dt);
  return { ...state, md, playing: md < tdFt };
}

// features/viewer/ui/layers/BitLayer.tsx
useFrame((state, dt) => {
  const drill = useDrilling.getState();
  const well = useSurveyStore.getState().wells[drill.activeWell!];
  const next = advance(drill, dt, well.actual.at(-1)!.md);
  useDrilling.setState({ md: next.md, playing: next.playing }); // no component
subscribed to md re-renders
  bit.current.position.copy(toScene(atMd(well.actual, next.md)!, tmp));
});
```

2. Geometry is rebuilt on a content key, never per frame.

```
// features/viewer/ui/layers/ActualLayer.tsx
const reached = useDrilling((s) => stationsReached(well.actual, s.md)); // changes
~every 95 ft
const curve = useMemo(
  () => new CatmullRomCurve3(densify(well.actual.slice(0, reached)).map((p) =>
toScene(p))),
  [well.version, reached],
);
return (
  <mesh>
    {/* new args → R3F builds a new geometry and disposes the old one */}
    <tubeGeometry args={[curve, Math.min(1400, curve.points.length * 3), 0.42, 10,
false]} />
    <meshStandardMaterial color={color} />
  </mesh>
);
```

3. Panels read fast values on a timer.

```
const md = useThrottledStore(useDrilling, (s) => s.md, 200);
const tip = atMd(well.actual, md);
```

Three traps the old repo fell into, now blocked:

- Calling `useDrilling()` with no selector subscribes to `md` and re-renders every frame. Always select fields (`useShallow` for several).
- Fitting the camera in an effect that depends on wells data snaps the view back every frame. `CameraRig` fits only when the view button or the well set changes.
- `material.dispose()` does not free textures. Let R3F own objects declared in JSX; dispose `<primitive>` objects yourself in an effect cleanup, as `BitLayer` does.

State library choices: **Zustand** for client state (surveys, drilling, settings), **TanStack Query** for server state once the API exists, **zod** at every boundary. No Redux or Context for fast-changing values.

Entra ID setup

Use **two app registrations** in the company's workforce tenant: one for the API (defines scopes and roles) and one for the React SPA (signs users in, has no secret). Every user, internal or external, signs in through this tenant, so Entra ID is the single identity provider.

1. API registration — `wellbore-api`

Setting	Value
Supported account types	Accounts in this organizational directory only (single tenant)
Expose an API → Application ID URI	`api://<api-client-id>`
Expose an API → scope	`Surveys.ReadWrite` (delegated, admins and users can consent)
Manifest	`requestedAccessTokenVersion: 2` so tokens carry `aud = <api-client-id>`
App roles (member type Users/Groups)	`Wellbore.Viewer`, `Wellbore.Engineer`, `Wellbore.Admin`
Enterprise app → Properties	*Assignment required* = **Yes**, so only assigned users get tokens
Enterprise app → Users and groups	Assign security groups to roles, not individuals

2. SPA registration — `wellbore-spa`

Setting	Value
Supported account types	Single tenant
Platform	**Single-page application** (enables auth code flow with PKCE)
Redirect URIs	`http://localhost:5173/` and each deployed origin, e.g. `https://wellbore.agenticnex.com/`
Front-channel logout / post-logout URI	Same origins
Certificates & secrets	**None.** A browser app cannot keep a secret
API permissions	`wellbore-api` → `Surveys.ReadWrite`, then *Grant admin consent*
App roles	The same three values as the API, assigned to the same groups (the UI reads them from the ID token)

Role model

Role	Can do
`Wellbore.Viewer`	View trajectories, geology, events, construction

Role	Can do
`Wellbore.Engineer`	Viewer + upload surveys, add live surveys, edit formations, events, construction
`Wellbore.Admin`	Engineer + manage wells, limits and configuration

Roles defined on the API registration appear in the **access token** the API receives; the API enforces them. The SPA copies of the roles only decide what the UI shows. When the API is live, replace the SPA copy with a `GET /api/me` call that returns the user's roles, so assignments live in one place. Per-well or per-asset permissions, if needed later, belong in the API's own database keyed on `tid` + `oid`, not in Entra.

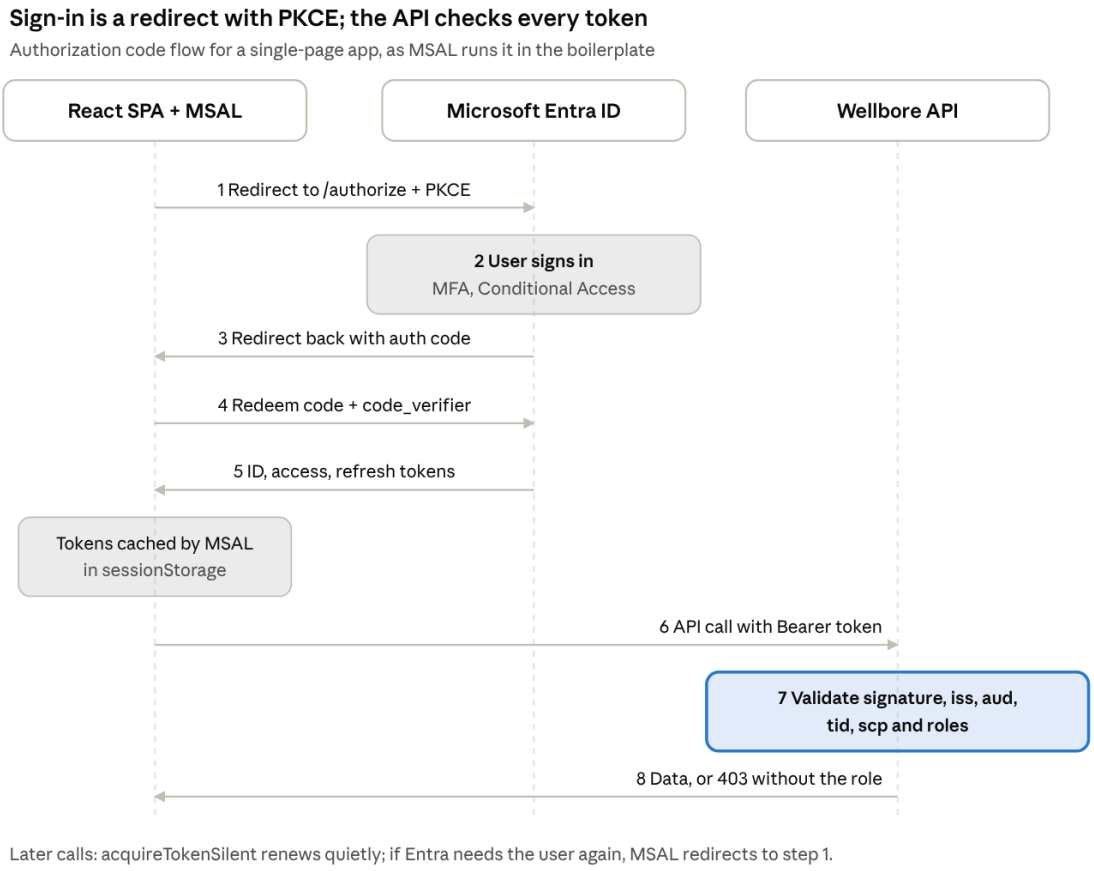
External users (partners, contractors, client engineers)

- Invite them as **B2B guests** into the workforce tenant. They sign in with their own organisation's account, a Google account or an email one-time passcode, and get roles exactly like employees.
- Use a separate **External ID external tenant** only if the app is later sold to customers who self-register. That changes the authority URL (`*.ciamlogin.com`), which the boilerplate reads from env.

Tenant policies (set by the Entra admin, not in code)

- Conditional Access: require MFA for all users of `wellbore-spa`; require a compliant or hybrid-joined device for `Wellbore.Admin`.
- Block legacy authentication; set sign-in frequency per company policy.
- Review guest access quarterly with Access Reviews.

Sign-in flow



sign-in and API call · 8 steps, 3 parties

The browser never holds a secret: PKCE proves that the app redeeming the code is the one that asked for it. Steps 1 to 5 happen once per session; step 6 repeats for every API call, and step 7 is the only place permissions are enforced.

Auth in the React app

The boilerplate uses `@azure/msal-browser` 5.25 and `@azure/msal-react` 5.7 (current majors as of this guide), wrapped behind one `AuthService` interface. Feature code only ever sees `useAuth()`, `<Authorized>` and `useApi()`.

1. Initialise before React renders — otherwise the first render sees no account and starts a second sign-in.

```
// src/shared/auth/entraAuth.tsx (called from main.tsx)
const pca = await createStandardPublicClientApplication(buildMsalConfig(env)); //
initialised instance
const redirect = await pca.handleRedirectPromise();                              //
back from Entra?
if (redirect?.account) pca.setActiveAccount(redirect.account);
```

```
else if (!pca.getActiveAccount()) {
  const [first] = pca.getAllAccounts();
  if (first) pca.setActiveAccount(first);
}
```

2. Configuration

```
// src/shared/auth/msalConfig.ts
auth: {
  clientId: env.VITE_ENTRA_SPA_CLIENT_ID,
  authority: env.VITE_ENTRA_AUTHORITY,          //
https://login.microsoftonline.com/<tenant-id>
  redirectUri: env.VITE_ENTRA_REDIRECT_URI,
  postLogoutRedirectUri: env.VITE_ENTRA_REDIRECT_URI,
},
cache: { cacheLocation: BrowserCacheLocation.SessionStorage }, // tokens die with the
tab
system: { loggerOptions: { piiLoggingEnabled: false, logLevel: LogLevel.Warning } },
```

All values come from env and are validated with zod at start-up; a missing client ID fails with a readable message. `VITE_AUTH_MODE=mock` is refused at build time and at runtime in production.

3. Gate the whole app with redirect sign-in (no popups; they break on mobile and in Teams):

```
<MsalProvider instance={pca}>
  <AuthContextProvider service={authService}>
    <MsalAuthenticationTemplate interactionType={InteractionType.Redirect}
authenticationRequest={loginRequest}>
      <App />
    </MsalAuthenticationTemplate>
  </AuthContextProvider>
</MsalProvider>
```

4. Tokens for the API — silent first, redirect only when Entra needs the user (expired session, MFA step-up, consent):

```
async getAccessToken() {
  const account = pca.getActiveAccount()!;
  try {
    return (await pca.acquireTokenSilent({ scopes: [env.VITE_API_SCOPE], account
})).accessToken;
  } catch (error) {
    if (error instanceof InteractionRequiredAuthError) {
      await pca.acquireTokenRedirect({ scopes: [env.VITE_API_SCOPE], account });
      throw new AuthRedirectError();
    }
    throw error;
  }
}
```

`createApiClient` calls this for every request, adds `Authorization: Bearer …` and an `X-Correlation-Id`, and validates the JSON with a zod schema. Features call `useApi().get('/wells', WellListSchema)`.

5. Roles in the UI

```
// shared/auth/roles.ts — one map from permission to minimum role (Admin ⊃ Engineer
⊃ Viewer)
```

```
export const PERMISSIONS = {
  'survey:upload': 'Wellbore.Engineer',
  'well:configure': 'Wellbore.Admin',
  // …
} as const;

// in any component
<Authorized permission="survey:upload" fallback={<p>View-only access.</p>}>
  <UploadButtons />
</Authorized>
```

`toRoles()` keeps only the three known role values from the token and ignores anything else. Components ask for a permission, never a role name, so changing who can do what is a one-line edit.

6. Local development and tests — `VITE_AUTH_MODE=mock` swaps in `createMockAuth(role)`, which implements the same interface. Component tests render `<AuthContextProvider service={createMockAuthService('Wellbore.Viewer')}>` to check role gating without a tenant.

Do not

- Store or copy tokens yourself, or put them in `localStorage` by hand.
- Decode the access token in the SPA; it belongs to the API.
- Use `loginPopup` as the default, or call MSAL from a feature.
- Treat a hidden button as protection.

Backend token validation

The API is the security boundary. Every request must pass these checks before any handler runs; the SPA never validates access tokens itself.

Check	Rule
Signature	Verify against the tenant's JWKS keys (`https://login.microsoftonline.com/<tenant-id>/discovery/v2.0/keys`), cache keys, refresh on unknown `kid`
Issuer `iss`	`https://login.microsoftonline.com/<tenant-id>/v2.0`
Audience `aud`	The API's client ID (v2 tokens)
Lifetime	`nbf` ≤ now < `exp`, allow ≤ 5 min clock skew
Tenant `tid`	Equals your tenant ID; reject anything else
Delegated scope `scp`	Contains `Surveys.ReadWrite`
Role `roles`	Contains the role the endpoint needs, e.g. `Wellbore.Engineer` for `POST /surveys`
User key	Store and authorise by `tid` + `oid`; never by `email`, `upn` or `preferred_username`

Use a maintained library rather than hand-rolled JWT code: `Microsoft.Identity.Web` for .NET, `jose` with the tenant JWKS endpoint for Node, `fastapi-azure-auth` or `PyJWT` + JWKS for Python. Return `401` for a missing or invalid token and `403` for a valid token without the role.

Also on the API: CORS allow-list of the SPA origins only, rate limits per `oid`, and an audit log of every write with `oid`, well, MD range and timestamp.

Boilerplate

`wellbore-viewer-boilerplate.zip` is a ready repo with one commit. From a clean unzip, `npm ci && npm run check` passes: strict type-check, ESLint with zero warnings, 53 tests, and a production build.

Run it

```
nvm use                          # Node 24
npm ci
cp .env.example .env.local       # set VITE_AUTH_MODE=mock for a first run without a tenant
npm run dev                      # http://localhost:5173 — loads the 3 sample laterals and starts drilling L1
```

What already works

- Sign-in with Entra ID (redirect, PKCE, sessionStorage cache), mock mode for local dev, role-gated upload controls.
- Minimum-curvature engine with explicit tie-in, arc interpolation, densify, junction detection, survey QA.
- CSV / TSV import in a Web Worker with header detection, row-level error reporting, 10 MB and 200,000-row caps.
- Only the selected wellbore drills; speed, pause, restart; live feed; plan-vs-actual deviation bands.
- 3D view: plan tubes per well, drilled tube rebuilt per station, bit and tip segment moved per frame, 3D / Plan / Section views, auto-rotate, light and dark themes from the same tokens as the page.
- ft / m switch with DLS in °/100 ft or °/30 m; per-viewer settings persisted.
- Zod schemas and pure rules already in place for formations, 21 event types and well construction (`formationAt`, `holeAt`, casing-fit and cement checks).

Stack (pinned)

Concern	Choice
Language / build	TypeScript 6.0 strict, Vite 8 (Rolldown), React 19.3
3D	three 0.186, @react-three/fiber 9.8, @react-three/drei 10.7 (lazy-loaded chunk)
State	Zustand 5 (client), TanStack Query 5 (server, ready for the API)
Validation	zod 4
Identity	@azure/msal-browser 5.25, @azure/msal-react 5.7

Concern	Choice
Parsing	Papa Parse 5.7 in a Comlink worker; XLSX deliberately omitted (see Migration)
Routing	React Router 8
Tests	Vitest 5 + Testing Library + jsdom; Playwright 1.64 (Chromium + WebKit)
Quality	ESLint 10 + typescript-eslint + react-hooks (React Compiler rules), Prettier

Tests that matter

- Known-answer MCM tests: vertical, straight inclined, constant 3°/100 ft build landing exactly one radius down and out, horizontal turn, tie-in.
- The reference sample data: 151 plan and 160 actual stations per lateral; L1 lands at 4,000 ft + R TVD.
- Role gating: a Viewer sees no upload controls, an Engineer does.
- Config: mock auth refused in production; every missing Entra value named.

Environment variables

Variable	Example	Notes
`VITE_AUTH_MODE`	`entra`	`mock` only for local dev
`VITE_ENTRA_TENANT_ID`	GUID	Directory (tenant) ID
`VITE_ENTRA_SPA_CLIENT_ID`	GUID	`wellbore-spa` Application (client) ID
`VITE_ENTRA_AUTHORITY`	`https://login.microsoftonline.com/<tenant-id>`	`*.ciamlogin.com` for an External ID tenant
`VITE_ENTRA_REDIRECT_URI`	`http://localhost:5173/`	Must match an SPA redirect URI exactly
`VITE_API_SCOPE`	`api://<api-client-id>/Surveys.ReadWrite`	From `wellbore-api` → Expose an API
`VITE_API_BASE_URL`	empty	Empty = sample data only

None of these are secrets; a SPA has none. Still, keep `.env.local` out of git and set production values as pipeline variables.

Delivery

- `.github/workflows/ci.yml`: audit → typecheck → lint → format → coverage → build (with real Entra values from repo variables) → Playwright.
- `staticwebapp.config.json`: SPA fallback plus CSP, HSTS, nosniff, referrer and permissions policies for Azure Static Web Apps. Replace `<your-api-host>` in `connect-src`.

- PR template with the architecture checklist.

Verified here, and not yet verified

- Verified: type-check, lint, 53 unit, component and app-integration tests, production build, clean-install from the zip.
- Not run here: the Playwright browser tests (no browser download in this environment) and a sign-in against a real tenant. Run `npm run test:e2e` and a tenant sign-in on day one.

Migration and PR checklist

Do not patch the current `wellbore-trajectory-react` repo; move its history aside and continue in the boilerplate. Port one row per pull request, each with tests and a side-by-side check against the reference HTML. The full list is in `docs/MIGRATION.md`.

#	Port from the reference HTML	Lands in	Status
1	MCM, arc interpolation, `atMd`, CSV parsing	`features/survey`	Done
2	Selected-well drilling, speed, deviation bands	`features/drilling`	Done
3	ft / m, light / dark	`shared/units`, `shared/theme`	Done
4	XLSX upload	SheetJS official CDN build inside `parse.worker.ts`	To do
5	Bit label, TVD staff + reference line, MD markers	`viewer/ui/layers/ScaleLayer.tsx`	To do
6	Formations: editor, TVD walls, MD overrides, sleeve, strip log	`features/geology`	To do (types + rules done)
7	Engineering events: 21 symbols, editor, 3D markers, feed	`features/events`	To do (types done)
8	Well construction editor, 3D casing/cement, schematic	`features/construction`	To do (types + rules done)
9	Readout extras: hole at bit, last shoe, formation, last event	`drilling/ui/BitReadout.tsx`	To do
10	Persist formations, events, construction	API via `useApi()` + TanStack Query	To do

Canvas-drawn parts (lithology patterns, event icons, schematic) become plain functions in `lib/` that take a `CanvasRenderingContext2D`; the React component only owns the `<canvas>` and redraws when data changes.

Pull request checklist (also in `.github/pull_request_template.md`)

- ☐ Code sits in the right feature folder; `lib/` stays free of React and three.js
- ☐ Nothing per-frame goes through React state; selectors pick only what the component shows
- ☐ New `lib/` or `model/` logic has unit tests; numbers match the reference HTML
- ☐ Checked in light and dark, in ft and m, and at 390 px width
- ☐ Role-gated UI uses `<Authorized>` or `useAuth().can()`, and the matching API rule exists or is ticketed
- ☐ No MSAL import outside `shared/auth`; no secrets or tenant values committed
- ☐ `npm run check` passes

Sources

- Add app roles to your application and receive them in the token — Microsoft identity platform docs
- Secure applications and APIs by validating claims — Microsoft identity platform docs
- Workforce and external tenant configurations — Microsoft Learn
- Collaborate with guests using External ID in your workforce tenant (B2B) — Microsoft Learn
- MSAL React package — npm; API surface checked against the published 5.7.2 and msal-browser 5.25.1 type definitions
- Microsoft identity platform code samples — Microsoft Learn